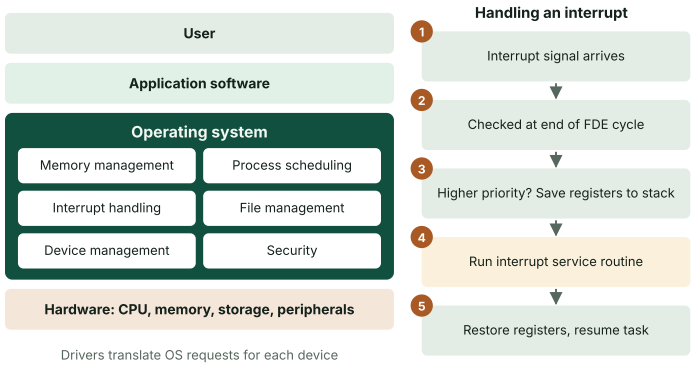
Operating systems and utility programs

FD14 FACT FILE

Specification: students should be able to

- Describe the purpose of an operating system and its main components: user interface, application interface, kernel and hardware interface
- Describe different types of operating system: single user, multi-user, multiprogramming, multiprocessing, multitasking and multithreading
- Explain memory management (paging and segmentation), process scheduling and the handling of interrupts
- Describe file management, device management using drivers, and the security functions of an operating system
- Explain the need for utility programs
- Describe common utility programs: data compression, file backup, archive software and disk defragmenters

Key words: operating system (OS), kernel, device driver, multi-user, time-slicing, multiprogramming, multitasking, multiprocessing, multithreading, deadlock, paging, segmentation, round robin, interrupt, interrupt service routine (ISR), utility program, file archiving, disk defragmenter



The operating system sits between applications and hardware and provides memory management, scheduling, interrupt handling, file and device management and security; an interrupt saves registers, runs the service routine and restores them

Overview: what an operating system is for

An operating system is the system software that manages every operation and resource of a computer. It sits between the hardware, the application software and the user, so a word processor can ask the OS to save a file rather than driving the disk itself. The OS is loaded first when the machine starts and stays in control while it runs. It can be pictured as four parts. The user interface takes input and presents information back, through a GUI or a command line. The application interface offers services that programs call and holds the device drivers that let the OS talk to each piece of hardware. The kernel is the core: it loads programs into memory, removes them when they finish, and shares memory, storage and processor time between them. The hardware interface links the CPU, memory and peripherals. Because an OS issues machine instructions, it must be built for a particular processor family. Alongside the OS sit utility programs, which together with the OS make up system software. Application software, by contrast, exists to help the user do a job, and always reaches the hardware through the OS.

Single user and multi-user systems

Operating systems can be classified by how many people and programs they support at once. A single user, single application OS lets one person run one program at a time and cannot distinguish between users; early desktop systems and simple embedded controllers worked like this. A single user, multitasking OS, the type found on most laptops, desktops and tablets, lets one person keep several programs open and switch between them, with the OS sharing resources among them. It needs more memory but is far more flexible. A multi-user OS allows many people to use one computer at the same time, each through their own terminal, and is typical of mainframes and large servers that may serve thousands of users concurrently. To make this work the OS shares processor time by time-slicing, giving each user's task a very short turn so that everyone appears to have exclusive use of the machine. It must also manage memory, storage and data requests so that every user gets enough, and it must keep each user's files private from all the others through accounts, passwords and access rights. Security and fair allocation are therefore the two responsibilities that most distinguish a multi-user OS.

Multiprogramming, multitasking, multiprocessing and multithreading

These four terms describe how an OS lets several programs share one computer. In multiprogramming, more than one program is resident in main memory, but only one executes at any instant. When the running program has to wait, typically for input or output, the OS switches the CPU to another program, so the processor is kept busy and the user sees an illusion of parallel running. Multitasking means essentially the same thing in a more general sense: tasks share the CPU and memory, and the OS allocates resources and switches between them, often using time-slicing. Multiprocessing is different because it uses two or more processors or cores, so work can genuinely run in parallel; its main benefit is speed. A system can be both multiprogramming and multiprocessing. Multithreading lets a single process run several threads, each a separate path of execution that shares the process's memory and code. A web server, for example, can handle many visitors using one copy of the program, with a thread per request. Threads are efficient, but if two threads update shared data at the same time results can be corrupted, and if each waits for a resource the other holds, both stop forever: a deadlock.

Memory management and scheduling

Memory management means allocating RAM to each program when it loads, keeping programs apart so one cannot overwrite another, and reclaiming memory when a program ends. When RAM is full the OS uses virtual memory, moving data that is not currently needed to secondary storage. Paging divides memory into equal, fixed-size pages; a page table maps each page of a program to any free frame in RAM, so a program need not be stored in one continuous block, although a part-filled final page wastes some space. Segmentation divides a program into variable-sized logical units such as a subroutine or a data structure, recorded in a segment table with a base address and length; this matches the program's structure but leaves awkward gaps between segments. Scheduling decides which process runs on the CPU and for how long. The aims are to keep the processor busy, maximise throughput, treat processes fairly and give interactive users quick responses. Round robin gives each process a fixed time slice in turn and sends unfinished processes to the back of the queue. First come first served, shortest job first and priority scheduling are alternatives; real-time systems rely on priorities so that urgent events are handled first.

Interrupts, devices, files and security

An interrupt is a signal telling the processor that something needs attention, such as a key being pressed, a printer running out of paper, a timer expiring or a hardware failure. Software can raise interrupts too, for example when a program tries to divide by zero. At the end of each fetch–decode–execute cycle the CPU checks for interrupts. If one is waiting and has a higher priority than the current task, the contents of the registers are saved on the stack, the correct interrupt service routine is run, and the saved values are then restored so the original task carries on as if nothing had happened. This is far more efficient than polling every device continually. Device management relies on device drivers: small programs, usually written by the hardware maker for each OS, that translate the OS's general requests into instructions a specific device understands. The OS also buffers and queues data, for example spooling print jobs. File management gives files names and folders, records where each is stored and controls who may read or change it. Security functions include user accounts, passwords, access rights, encryption, a firewall and logging, and they matter most on multi-user and networked systems.

Utility programs

Utility programs are system software tools that carry out specific housekeeping jobs to keep a computer efficient, secure and recoverable. They are needed because users and administrators must do these jobs regularly, and many are now built into the OS. A data compression utility encodes files using fewer bits so they take less storage and transfer more quickly; reducing size is a means to use resources better rather than a goal in itself. A backup utility makes exact copies of files or databases, usually on a schedule and ideally off-site, so that originals can be restored after hardware failure, accidental deletion or a ransomware attack. An archiving program combines many files into one archive file or a set of them, often compressing them, for long-term storage or transfer. It records metadata such as file names, lengths, timestamps and access control lists so each file can be rebuilt exactly, and an administrator decides what is archived, where, for how long and who can access it. A disk defragmenter rearranges the clusters of each file on a magnetic hard disk so they are contiguous, cutting the head movement needed to read them. It can take hours and should not be used on solid state drives.

T Key terms from the mark schemes

Operating system
System software that manages a computer's hardware and software resources and sits between the hardware, the application software and the user so that they can work together.
`Credited: Software that controls the hardware and provides a platform for applications`

Kernel
The core of the operating system, which loads and unloads programs in memory and allocates memory, storage and processor time.

Device driver
Software that allows the operating system to communicate with and control a specific hardware device.

Multi-user operating system
An operating system that lets many users use one computer at the same time, sharing its resources by time-slicing while keeping each user's data private.

Time-slicing
Sharing processor time by giving each task or user a short, fixed period of CPU time in turn.
`Credited: Time slice`

Multiprogramming
Holding several programs in main memory at once and switching the CPU between them, although only one executes at any instant.

Multiprocessing
Dividing work between two or more processors or cores so that it is executed truly in parallel.
`Credited: Parallel processing`

Multithreading
Running several threads, each a separate path of execution, within one process so they share its memory and code.

Paging
A memory management technique that divides memory into fixed-size pages, mapped to physical frames by a page table.

Segmentation
A memory management technique that divides a program into variable-sized logical segments such as modules and data structures.

Interrupt
A signal to the processor from hardware or software indicating that an event needs attention; the current task is suspended while an interrupt service routine runs.

Round robin
A scheduling algorithm in which each process is given a fixed time slice of the processor in turn.

Utility program
System software that performs a specific task to help manage, maintain or protect the computer, such as compression, backup or defragmentation.
Credited: Utility software

Disk defragmenter
A utility that rearranges the data on a magnetic hard disk so each file occupies contiguous clusters, reducing access time.

C C# example

Multithreading in C#

```
using System;
using System.Threading;

int counter = 0;
object key = new object();

Thread a = new Thread(AddMany);
Thread b = new Thread(AddMany);
a.Start(); b.Start();
a.Join(); b.Join();          // wait for both threads
Console.WriteLine(counter);  // 200000

void AddMany()
{
    for (int i = 0; i < 100000; i++)
    {
        lock (key) { counter++; }  // one thread at a time
    }
}
```

Exam focus

Short questions such as 'State the purpose of an operating system' earn a mark for managing hardware and software resources and another for acting as the interface between hardware, applications and the user. 'Describe' questions on OS types expect two or three precise features each: for a multi-user OS, mention many users at once, their own terminals, time-slicing, managing resources fairly and keeping users' data private. For the four 'multi' terms, avoid circular answers like 'multitasking does many tasks'. Say that in multiprogramming several programs are in memory but only one executes at a time, that multiprocessing uses more than one processor for true parallelism, and that multithreading runs several threads within one process sharing its resources. Interrupt answers need the sequence in order: checked at the end of the cycle, priority compared, registers saved to the stack, ISR run, registers restored. In utility questions, give the benefit as well as the action: a defragmenter makes clusters contiguous so files are read faster. Common mistakes are calling a device driver hardware, saying that deleting a file erases its data, and recommending defragmentation for an SSD. Round-robin traces should be set out as a table of time intervals.

? Check yourself

1. State two purposes of an operating system. [2]
2. Describe the role of the kernel in an operating system. [2]
3. A health trust's patient records system runs on a multi-user operating system used by hundreds of staff at the same time. Describe how the operating system makes sure that each user gets fair use of the system and that their data is kept private. [4]

Answers: 1. Any two from: manages the computer's hardware resources (e.g. memory, processor, devices) (1) / provides an interface between the user and the hardware (1) / provides a platform/environment for application software to run and access the hardware (1) / manages files and security (1) 2. Any two from: core of the OS (1) / loads programs into memory and unloads them when finished (1) / allocates resources such as memory and secondary storage (1) / schedules use of the processor (1) 3. Any four from: time-slicing shares processor time between users' tasks (1) / so each user seems to have the computer to themselves (1) / memory and storage are allocated so each user has sufficient resources (1) / data requests are queued/managed (1) / each user has a user account and password to authenticate (1) / access rights restrict each user to their own files/records (1) / one user's files are kept private from others (1)